Memory-Aware Agent Edge Serving: Phase 1 Baseline and Measurements

Yunxuan Li [1] **Max Huang** [1]

Abstract

On a memory-constrained machine, how should a server admit and schedule concurrent agent tasks so that completed work increases while memory stays bounded? We build a runner-owned FIFO baseline over llama.cpp with a seeded workload of short, long-context, long-generation, and multi-step agent tasks, and measure 45 runs (17.9 h) on a CPU-only laptop. The configured KV buffer is allocated at startup, so process `footprint` (a lower bound of 3.75–3.81 GiB) does not track load and the working budget is KV capacity. Under a static partition at λ=0.088 task/s, raising concurrency from 4 to 8 lifts goodput from 0.015 to 0.052 task/s, but short-task completions rise from 22/116 to 111/116 while long-context completions fall from 62/62 to 0/62. Overload failures are queueing timeouts; compute contention is their likely cause but is not isolated. We keep c=4 as a provisional baseline: at low load it meets our selection rule, while c=2 is the one candidate not yet compared there. Retained context covers 84% of agent-continuation prompts at low load and 54% in overload. These findings motivate KV-budget admission with size-aware ordering for Phase 2.

1. Problem and Setup

Question. Admitting too much agent work raises latency or exhausts memory; admitting too little idles the machine. Phase 1 builds the FIFO baseline and measures what limits it.

Environment (all pinned). MacBook Air (M4, 16 GB, 10 cores) on AC power; polled every 30 s, the OS recorded no thermal warning in any run. CPU-only `llama-server` (llama.cpp `1b29621`, `b11741`, Metal off), Llama-3.2-3B-Instruct Q4_K_M (sha256 `6c1a2b41…28ff`), `-t 8 --cache-ram 0`. The memory budget B is the total KV-token count (`-c 16384`, 1792 MiB at 112 KiB/token). *Series S* fixes B and sets `-np`= c, so each slot holds B/c tokens; *series U* uses one shared pool (`-kvu -np 8`).

Workload. A seeded, byte-reproducible Dev workload (seeds 1–3 here; Eval seeds 101–105 with a different mix are untouched): 100 tasks, mix 40/20/20/20 of *short* (256 in/64 out tokens), *longctx* (3000/64), *longgen* (256/512), and *agent* (about 800-token context, 3–6 steps of 128 tokens, tool delay 0.2–3 s). Arrivals are open-loop Poisson. Each task embeds a random needle checked programmatically (single-stream pass rate 100/95/100/95%). The runner owns a step-level FIFO with $c \leq$ `-np`; tool delays hold no slot.

Metrics and rules. Per-request and per-task logs, a 0.5 s system sampler, and a per-run manifest (git sha, versions, command line, workload hash). Deadline is $5\,T_{\text{solo}}$ (frozen); task timeout is $\min(300, 10\,T_{\text{solo}})$ s with active disconnect. Failures and timeouts stay in every denominator; the first 3 tasks of each run are warm-up and excluded. Queue statistics include timed-out tasks unless marked otherwise, and a task that times out while queued is censored at its timeout. Peak memory is macOS `footprint`, because RSS is unreliable here (idle RSS of one configuration ranged 689–4678 MiB across launches); `footprint` omits about 294 MiB of memory-mapped weights, so it is a lower bound. We did not sample CPU utilization.

2. Results

All 45 Dev runs passed validity checks: concurrency sweep E1 ($c \in \{1, 2, 4, 8\}$ in S, $\{4, 8\}$ in U, λ=0.088 task/s), tool delay E3 (0–3 s), `cache_prompt` on/off E4 ($c \in \{1, 4\}$), and arrival rate E5 (c=4), each with 3 seeds. Appendix B gives mean±sd for every configuration.

F1: Process memory is set at launch, so the working budget is KV capacity. The configured KV buffer is allocated at startup, and the per-run peak `footprint` stays within 3840–3905 MiB in all 45 runs (Fig. 1c), across c, λ, and series. Doubling `-c` raised idle `footprint` by 1795 MiB,

[1]CSE 291A (Fall 2026), UC San Diego; course project, Option 1, Phase 1 report. Correspondence to: Yunxuan Li <yul386@ucsd.edu>, Max Huang <mah067@ucsd.edu>.

Preprint. November 4, 2026.

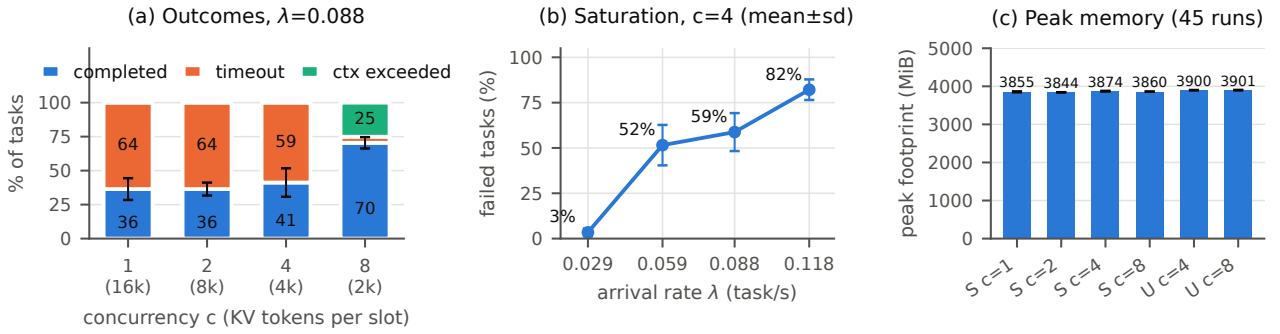


Figure 1. (a) Outcome of steady-state tasks (3 seeds, 291 tasks per bar); whiskers show the sd across seeds of the completed share. (b) Failure rate at c=4 versus arrival rate. (c) Peak `footprint` for every configuration (S: np=c; U: shared pool). Data: `results/analysis/batch/*.csv`.

matching the 1792 MiB KV difference (one launch each). In these configurations, memory pressure therefore appears as slot capacity and admission, not as process growth. This bounds one process and is a lower bound; we did not measure system-wide memory pressure.

F2: A static partition trades concurrency against workload coverage. At λ=0.088, increasing concurrency from 4 to 8 raised aggregate goodput from 0.015 to 0.052 task/s. Short-task completions rose from 22/116 to 111/116, while long-context completions fell from 62/62 to 0/62 because each slot could hold only 2048 tokens (Table 1). Aggregate goodput therefore hides a loss of workload coverage. Total completions rose from 120/291 to 205/291, and these runs cannot separate the effect of higher concurrency from the capacity freed by rejecting longctx within seconds.

The cap is visible at the step level. All 11 agent failures are the six-step tasks, each failing at step 6, whose prompt measures about 2257 tokens at c=4. Their step 5 (prompt 1947–1951) had already stopped at 97–101 of 128 output tokens when the slot filled. All 12 completed five-step tasks also ended this way (75–85 of 128 tokens), yet they count as `ok` and 11 pass the needle check. Excluding them lowers c=8 goodput from 0.052 to 0.049 task/s. No other run produced a short output, so output tokens must be checked alongside the needle.

Provisional baseline. We do not treat c=8 as a valid baseline. Our original selection rule (p99≤deadline, at most 5% failures, else highest goodput) picks c=8, because p99 covers only completed tasks and c=8 rejects the slowest class: this is survivorship bias. We replaced it *after seeing the data* with no `ctx_exceeded` and at most 10% failures, so this choice is not independent evidence. All c are overloaded at λ=0.088: excluding c=8, c=4 has the highest goodput on all three paired seeds but still fails 59% of tasks, so that rate cannot select a baseline.

At λ=0.029, the rule leaves one comparison open. c=8 is out because it rejects longctx. c=4 qualifies, failing 3.4 ± 2.1% of tasks. c=1 exceeds the limit, but only on one earlier run (seed 1, `-np 4`; 18/97 failed versus 5/97 for c=4 on the same trace). c=2 **was not run at this rate**, so c=4 is *provisional*, not tuned. The gap is bounded without a model: c=4 already delivers 241 of 291 tasks on time and passing the needle check (goodput 0.025 ± 0.003 task/s), so no configuration can add more than 50 tasks on these traces (+21%). That margin is real, and 17 of the 50 are late short tasks that lower concurrency might rescue, so we do not claim c=4 beats c=2. F1–F5 report the configurations as measured and do not depend on which is selected. Measuring c=2 is the first step of M1 (Appendix A), under a rule fixed before those runs; if it wins, Phase 2 policies A and B use it.

Shared pool. Series U has no admission control and was not better than S at c=4 (65% vs. 59% failed, sd about 10 points). Pool pressure did occur: the six U server logs record 12 "failed to find free space" events, each paired with a purge of retained tokens from an idle slot (up to 3064 tokens). None reached the next stage, which source reading says returns HTTP 500 to *all* running requests. That failure mode remains unmeasured, so the case for avoiding it rests on the source, not on our data.

F3: Overload failures are queueing timeouts; compute contention is the likely cause. *Measured:* at c=4, failures rise 3%→52%→59%→82% across λ=0.029 . . . 0.118 (Fig. 1b), so saturation lies between 0.029 and 0.059 task/s, well below the μ=0.098 estimate; the exact point is unmeasured. Across the same range, the share of task time spent queued goes from 10% to 72% (59% at λ=0.088; Fig. 2a). At λ=0.088, 94 of 116 short tasks (81%) timed out and 84 of those never reached a slot. Their median wait of 31.9 s nearly exhausts the 32 s limit, while the 22 that completed waited a median of 1.5 s (Table 2).

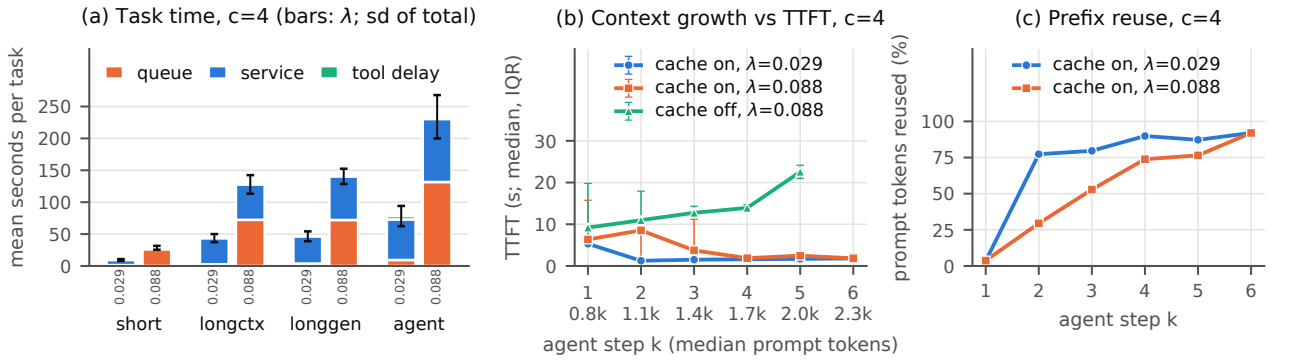


Figure 2. (a) Mean time per steady-state task, including timed-out tasks, at c=4; each pair of bars is λ=0.029 and 0.088. (b, c) Request-level view of completed agent steps, 3 seeds pooled. TTFT excludes runner queueing. Later steps have few samples in overload (n for cache on: 50, 34, 22, 11, 6, 3; cache off: 37, 22, 11, 4, 2) and only tasks with at least k steps reach step k. Data: `tables/derived_numbers.json`.

Table 1. Completed tasks by type (S, 3 seeds pooled). Failures are task timeouts unless marked "ctx" (`error_ctx_exceeded`). Last row: sd across seeds of the per-run completed count.

	λ=0.088				λ=0.029
Type (total)	c=1	c=2	c=4	c=8	c=4
short (116)	8	9	22	111	110
longctx (62)	62	62	62	0 (62 ctx)	62
longgen (60)	27	23	19	52	58
agent (53)	9	12	17	42 (11 ctx)	51
all (291)	106	106	120	205	281
per-seed sd	7.8	4.6	10.1	4.0	2.1

Table 2. Median seconds by type (S, 3 seeds pooled). Q_{all}: queue wait over all tasks, including timeouts; Q_{ok}: queue wait of completed tasks; S: service time of completed tasks, excluding tool delay. Medians over completed tasks carry survivorship bias under overload.

	c=4, λ=0.029			c=4, λ=0.088			c=8, λ=0.088		
Type	Q_{all}	Q_{ok}	S	Q_{all}	Q_{ok}	S	Q_{all}	Q_{ok}	S
short	0.0	0.0	4.3	31.9	1.5	6.1	0.0	0.0	8.3
longctx	0.0	0.0	35.1	72.0	72.0	44.6	0.0	–	–
longgen	0.0	0.0	29.6	75.5	10.0	67.8	0.0	0.0	44.9
agent	0.0	0.0	46.7	142.3	33.9	85.4	0.0	0.0	68.0

Inferred: service time inflates under contention. Even at λ=0.029, median longgen takes 29.6 s vs. T_{solo} 15.95 s, and longctx 35.1 vs. 26.4 s. Series U, which rejects no task class, completes *fewer* tasks at c=8 than at c=4 (70 vs. 101 of 291, lower on every seed). A fixed-request benchmark (16 requests of 128 tokens, 2 rounds) gives 38.3 tok/s at c=4 vs. 42.8 at c=8, but c=4 varied by 20% between rounds, so it does not establish saturation alone. These results point to compute contention, but without CPU utilization data we cannot confirm it or separate CPU from memory bandwidth. Short tasks waiting behind 26–45 s longctx tasks are consistent with head-of-line blocking, which we have not isolated.

F4: Context growth is cheap when the prefix survives, and overload makes it fragile. Agent prompts grow from about 800 to 2257 tokens over 6 steps. TTFT excludes runner queueing, so it remains informative in overload. With `cache_prompt` off, median TTFT grows with the prompt (9.2 s at step 1 to 13.9 s at step 4; c=4, λ=0.088). With it on at λ=0.029, TTFT falls from 5.3 s at step 1 to 1.3–1.8 s at steps 2–6 as the prompt nearly triples (Fig. 2b). Prompt reuse at steps $\geq$ 2 is 84% at c=4, λ=0.029, then 54% at λ=0.088 and 24% at c=1, λ=0.088, versus 0% off (Fig. 2c). We observed one cause directly: during a tool delay, an unrelated short request was routed by prefix similarity (0.125, above the 0.10 threshold) to the agent's slot and overwrote its prefix.

At the task level, paired on/off differences favor caching (p50 −27 s at c=1, −36 s at c=4; failures −9.6/−18.6 points), but both sides are overloaded and there are only 3 pairs with large sd (14 and 51 s), so only the direction is reliable. Lacking cache-off runs at low load, we cannot state a clean effect size.

F5: The effect of idle periods is unresolved. Tool delays hold no slot here, so idle time can only act through retained context. E3 (tool delay 0–3 s, λ=0.088) shows no failure trend (61, 61, 60, 63%, sd 7–10 points) because overload swamps it. Continuation-prompt reuse is 54, 52, 52, and 46% across delays of 0, 0.5, 1, and 3 s, but seeds differ by more than that (37–56% at 3 s), so this is not evidence of an effect. Answering this needs E3 at a non-overloaded rate (Appendix A).

3. Phase 2 Direction

FIFO with a fixed c cannot see the one resource that differs across tasks: KV tokens. A static partition rejects or truncates long contexts at high c; a low c serializes everything behind long steps (F2, F3). We will build *one* mechanism: **KV-budget admission with size-aware ordering on the shared pool.** Each ready step has a known cost (prompt tokens plus `max_tokens`). The scheduler admits it only if the sum of reserved tokens stays within B, so concurrency adapts to task size (many short steps, few long ones). Among ready steps it prefers cheaper and continuing agent steps, relieving F3. A cache-aware extension pins a continuation to its previous slot via the request-level `id_slot` (present in the source; to be verified on the OAI endpoint) and is compared against the fixed policy (`cache_prompt` on) and off (F4). The scheduler can verify that reservations stay within B; that the runtime then never exhausts the pool is a stress-test target, not an assumption, because retained prefixes also occupy it.

Evaluation plan. Four policies on identical arrival sequences separate the sources of any gain:

	Policy	Compared with
A	Static partition, FIFO, tuned fixed c	Course baseline
B	Shared pool, FIFO, same fixed c	A: KV layout alone
C	Shared pool, KV-budget admission, FIFO order	B: admission gain
D	Shared pool, same admission, size-aware order	C: ordering gain, cost to long tasks

We use the Eval workload (seeds 101–105, different mix), sweeping λ from under- to overload, including the underloaded case where no gain is expected. We report completed tasks, p50/p99 latency, throughput, peak memory, failure rate, and quality pass against the 100/95/100/95% baseline, plus actual output tokens, so gains cannot come from skipped tasks or shortened output (F2). We also report policy overhead and long-task fairness through both longctx completion rate and p99, since p99 covers completed tasks only. Timeline: Appendix A; full metrics: Appendix B.

4. Limitations

Single laptop, CPU-only, one model: numbers may not transfer to AWS or Metal. Three seeds of 100 tasks, and most E1/E3/E4 points are in overload, so effect sizes are weak; the λ grid came from a capacity estimate that proved too high. Not yet measured: c=1, 2 at low load (beyond the one c=1 run), E3/E4 at low load, CPU utilization, and pool exhaustion. Background load (XProtect) was not excluded.

Stability. No OS thermal warning does not rule out frequency scaling on a fanless machine, and we did not remeasure T_{solo} before and after the batch. As an after-the-fact proxy, requests that ran alone show no progressive slowdown: median short takes 2.87 s in the first 3 h (n=16) and 3.16 s in the last 3 h (n=6), vs. T_{solo} 3.19 s. Longctx does show a transient: 28.5, 33.2, 34.3, then 27.4 s across 3-hour bins through hour 12 (n=46, 52, 12, 10). The cause is unknown, and the bins mix configurations.

Appendix A. Timeline and Milestones

Phase 1 (completed; dates from the git history and lab notebook).

Date	Milestone
Oct 18	Option 1 chosen; course and project requirements read.
Oct 19	Sample reports reviewed; pinned llama.cpp and model; memory baseline; capacity calibration; metrics fixed. Runner built test-first against a fake server: workload generator, FIFO, Poisson arrivals, timeouts, agent tasks, sampler, manifest, analysis command.
Oct 30	Pre-batch checks: `-t 8` frozen after interleaved comparison; T_{solo} re-measured on AC power (23–29% faster, so λ grid and deadlines recomputed); c=8 throughput; cold start.
Oct 31	Real-server smoke test (30 tasks, logs complete); single-stream quality baseline and frozen deadlines (Dev workload hash fixed); resumable batch runner with 30-task pilots.
Oct 31–Nov 1	45-run E1–E5 batch (17.9 h wall clock, no failed runs or OS thermal warnings); analysis; correction that c=8 looks best only because it rejects longctx. First version of this report written Nov 1.
Nov 4	Revised report: per-type queue and service breakdown, step-level analysis, and output-token check, all re-derived from the existing logs. No new server runs.

Phase 2 plan (due December 10, 2026).

Window	Milestone and exit criterion
Nov 5–8	**M1, baseline (33 Dev runs, about 29 h unattended, plus a short pool stress test).** First close the open comparison: c=2 at λ=0.029 on seeds 1–3 (about 3 h). On the same Dev traces, then run c=1 at λ=0.029 and c=1, 2, 4 at λ=0.045; whether 0.045 is overloaded is decided by the measurement. Repeat E3 (delays) and E4 (cache off, c=1, 4) at λ=0.029. Deliberately exhaust a U pool with more longctx and higher concurrency, recording the failure and recovery cost. Measure T_{solo} before and after the batch; log CPU utilization. Verify `id_slot` pinning on the OAI endpoint. *Rule, fixed before these runs:* a configuration must admit all four task types and fail at most 10% of tasks; among those, take the highest goodput, with ties within 5% going to the smaller c. If none qualifies, report that and keep the baseline provisional rather than revising the rule. *Exit:* baseline c selected by this rule, which then sets c for policies A and B; if unresolved, A and B run at both c=2 and c=4.
Nov 9–15	**M2, mechanism.** Implement KV-budget admission and size-aware ordering behind the scheduling-policy interface; fake-server tests; 30-task pilots. *Exit:* reserved tokens never exceed the budget.
Nov 16–22	**M3, tune and freeze** on Dev seeds only; ordering ablation (C vs. D) to test the head-of-line explanation; freeze parameters.
Nov 23–Dec 1	**M4, Eval runs** (seeds 101–105; λ sweep and mix shift; policies A–D on identical arrivals; cache-aware vs. fixed cache policy; at least 3 repeats). Thanksgiving, Nov 26: runs unattended.
Dec 2–6	**M5, analysis.** Quality and output-token parity, overhead, fairness, no-benefit regimes, failure accounting.
Dec 7–10	**M6, report and artifacts.** Final report; clean-clone reproduction of all figures; tag and submit on Dec 10.

Appendix B. Performance Summary

Mean±sample sd over 3 seeds (Dev workload, warm-up excluded; `cache_prompt` on except E4). p50/p99 cover *completed* tasks only, so they flatter configurations that reject work (S, c=8 rejects all longctx). Queue = mean per-task queue wait over all tasks, including timeouts; Thr. = completed tasks/s over the steady-state window; Goodput counts only tasks meeting deadline and quality check; Fail = share of tasks not completed; Peak = max `footprint`, grouped by series and c.

Exp.	Configuration	p50 (s)	p99 (s)	Queue (s)	Thr. (task/s)	Goodput (task/s)	Fail (%)	Peak (MiB)
E1	S, c=1, λ=0.088	153 ± 15	242 ± 41	109 ± 7	0.028 ± 0.005	0.005 ± 0.003	64 ± 8	3855 ± 11
E1	S, c=2, λ=0.088	143 ± 2	247 ± 27	95 ± 7	0.028 ± 0.003	0.006 ± 0.001	64 ± 5	3844 ± 0
E1	S, c=4, λ=0.088	102 ± 44	254 ± 35	65 ± 11	0.033 ± 0.007	0.015 ± 0.007	59 ± 10	3874 ± 6
E1	S, c=8, λ=0.088	18 ± 4	142 ± 53	0 ± 0	0.063 ± 0.005	0.052 ± 0.008	30 ± 4	3860 ± 1
E1	U, c=4, λ=0.088	122 ± 40	265 ± 5	70 ± 11	0.027 ± 0.008	0.010 ± 0.006	65 ± 11	3900 ± 1
E1	U, c=8, λ=0.088	51 ± 9	210 ± 69	18 ± 9	0.020 ± 0.009	0.013 ± 0.010	76 ± 12	3901 ± 4
E4	S, c=1, cache off	180 ± 12	257 ± 2	120 ± 5	0.020 ± 0.005	0.002 ± 0.001	73 ± 5	3855 ± 11
E4	S, c=4, cache off	138 ± 17	258 ± 5	81 ± 4	0.017 ± 0.003	0.004 ± 0.001	77 ± 2	3874 ± 6
E5	S, c=4, λ=0.029	25 ± 3	170 ± 58	4 ± 3	0.029 ± 0.003	0.025 ± 0.003	3 ± 2	3874 ± 6
E5	S, c=4, λ=0.059	74 ± 35	275 ± 18	38 ± 9	0.028 ± 0.005	0.017 ± 0.006	52 ± 11	3874 ± 6
E5	S, c=4, λ=0.118	177 ± 26	261 ± 1	99 ± 5	0.017 ± 0.005	0.003 ± 0.001	82 ± 6	3874 ± 6
E3	S, c=4, delay 0 s	103 ± 39	242 ± 17	65 ± 9	0.031 ± 0.007	0.014 ± 0.007	61 ± 10	3874 ± 6
E3	S, c=4, delay 0.5 s	105 ± 38	233 ± 23	65 ± 9	0.031 ± 0.005	0.013 ± 0.007	61 ± 8	3874 ± 6
E3	S, c=4, delay 1 s	114 ± 22	248 ± 34	66 ± 9	0.031 ± 0.004	0.012 ± 0.002	60 ± 7	3874 ± 6
E3	S, c=4, delay 3 s	112 ± 20	237 ± 19	69 ± 8	0.029 ± 0.006	0.013 ± 0.003	63 ± 10	3874 ± 6

By task type. Pooled over 3 seeds; c=2 and λ=0.059, 0.118 are in `derived_numbers.json`. Good = within deadline and passing the needle check; Quality = completed tasks passing the needle check; Never disp. = timed out before reaching a slot; other columns as in Table 2.

Configuration	Type	Tasks	Compl.	Good	Quality	Timeout	Ctx	Never disp.	Q_{all} (s)	Q_{ok} (s)	S (s)
S, c=1, λ=0.088	short	116	8	1	7	108	0	108	31.9	18.1	3.8
	longctx	62	62	9	60	0	0	0	136.3	136.3	30.1
	longgen	60	27	7	24	33	0	18	142.5	97.0	19.0
	agent	53	9	2	8	44	0	0	243.3	148.4	40.8
S, c=4, λ=0.088	short	116	22	13	21	94	0	84	31.9	1.5	6.1
	longctx	62	62	29	60	0	0	0	72.0	72.0	44.6
	longgen	60	19	6	18	41	0	1	75.5	10.0	67.8
	agent	53	17	7	16	36	0	0	142.3	33.9	85.4
S, c=8, λ=0.088	short	116	111	90	108	5	0	0	0.0	0.0	8.3
	longctx	62	0	0	0	0	62	0	0.0	–	–
	longgen	60	52	42	49	8	0	0	0.0	0.0	44.9
	agent	53	42	36	40	0	11	0	0.0	0.0	68.0
U, c=4, λ=0.088	short	116	17	10	15	99	0	88	31.9	0.0	6.6
	longctx	62	56	15	55	6	0	0	85.8	86.9	65.6
	longgen	60	12	5	11	48	0	2	89.3	0.0	76.7
	agent	53	16	9	15	37	0	0	141.6	25.5	90.3
U, c=8, λ=0.088	short	116	28	15	26	88	0	24	8.1	0.0	11.6
	longctx	62	21	19	21	41	0	0	7.2	0.0	71.3
	longgen	60	6	4	6	54	0	0	7.2	0.0	68.3
	agent	53	15	9	14	38	0	0	12.5	0.0	113.0
S, c=4, λ=0.029	short	116	110	91	106	6	0	3	0.0	0.0	4.3
	longctx	62	62	60	61	0	0	0	0.0	0.0	35.1
	longgen	60	58	47	55	2	0	0	0.0	0.0	29.6
	agent	53	51	43	47	2	0	0	0.0	0.0	46.7

Reproduction. All 45 runs used repository commit `d23e96d` with a clean working tree. Source: `github.com/YunxuanLi0619/cse291-agent-serving`; raw runs: `results/raw/batch/`; every log-derived number in the text: `report/phase1/tables/derived_numbers.json`. From the repository root, with llama.cpp built at the pinned commit (`cmake -B build -DGGML_METAL=OFF -DCMAKE_BUILD_TYPE=Release`) and the model in `~/models`:

```
uv venv .venv --python 3.12 && uv pip install -r requirements.lock.txt
# Workloads are checked in; to regenerate one, start a server (it calibrates token lengths) and run:
scripts/start_server.sh 4 16384 --no-kv-unified & scripts/wait_health.sh
.venv/bin/python -m src.runner.gen_workload --mix configs/mix_dev.json --t-solo configs/t_solo.json \
  --seed 1 --n-tasks 100 --out workloads/dev/default_s1.json
# Run the batch (the driver restarts the server for each run and can resume after an interruption):
caffeinate -is scripts/run_batch.sh
# Rebuild the CSVs, figures, tables and this PDF from the raw logs (no server needed):
.venv/bin/python -m src.analysis --raw results/raw/batch --out results/analysis/batch
.venv/bin/python report/phase1/make_breakdown.py
.venv/bin/python report/phase1/make_figs.py
.venv/bin/python report/phase1/make_table.py
cd report/phase1 && tectonic phase1_report.tex
```